

RERC Review Portal — Project Context Pack

Project Context Pack

Generated May 28, 2026

0. Executive Summary

System Purpose

The **RERC Review Portal** is a web-based Research Ethics Review Committee (RERC) management system built for De La Salle University Manila. It streamlines the workflow for managing research ethics protocol submissions—from initial receipt through classification, review, and final decision—while providing mail-merge letter generation, SLA tracking, holiday management, reporting, account governance, panel management, assigned-only Research Assistant access, and protocol profile capabilities.

Tech Stack

Layer	Technology
Backend	Node.js 18+, Express 5.x, TypeScript 5.x
Frontend	React 18, Vite 5.x, TypeScript, React Router 6
Database	PostgreSQL (via Prisma ORM 7.x)
Libraries	Axios, docx (Word generation), json2csv, csv-parse
Infra	Local development (no CI/CD or containerization yet)

Highest-Risk Areas / Bottlenecks

- **Authentication has improved but still needs production review:** The app now has auth/session middleware, cookies, CSRF protection, forced-password-change handling, and account approval, but the deployment should still be reviewed before treating it as final enterprise auth.
- **Role checks must stay backend-enforced:** Sidebar visibility and route guards are useful UX, but direct URL/API access must continue to be blocked by backend middleware.
- **Assignment semantics are easy to confuse:** Assign assistant and Assign reviewer are separate features with different database paths and permissions.
- **CSV/file import security:** Seed script parses external CSV without comprehensive sanitization.
- **No automated tests running in CI:** Tests exist but no CI/CD pipeline.
- **Large monolithic route files:** Some route files exceed 800 lines.

Where to Start Reading the Code (Top 5 Files)

1. `backend/src/server.ts` — Express app entry point, middleware setup, route mounting
2. `backend/prisma/schema.prisma` — Complete data model (781 lines) with all enums and relationships
3. `backend/src/middleware/reviewerScope.ts` — Assigned-only and protocol-operator access guards
4. `frontend/src/pages/DashboardPageNew.tsx` — Main dashboard UI with route-driven queues
5. `docs/user-manual.md` — User-facing handoff manual for real workflows and RBAC behavior

May 28, 2026 RBAC Handoff Update

This context pack now reflects the current role-based behavior used in the deployed portal:

- **Panel Management** is Chair-only in the sidebar and protected route layer.
- **Account Management** is available to Chair/Admin, with Chair-only approval actions.
- **Assign assistant** gives a Research Assistant protocol-operator access through `Submission.staffInChargeId`.
- **Assign reviewer** creates review tasks through `Review.reviewerId`; it does not grant protocol-operator access.

- **Research Assistants are assigned-only users.** They can see a protocol if assigned as staff-in-charge or reviewer, but operational protocol controls require staff-in-charge assignment.
- **Direct URL access is guarded** by backend submission/project access middleware, not only by hidden UI buttons.
- **The handoff manual exists in both Markdown and PDF** at docs/user-manual.md and docs/user-manual.pdf.
- **The Neon quota runbook exists in both Markdown and PDF** at docs/neon-quota-runbook.md and docs/neon-quota-runbook.pdf.

1. Repository Map

```

rerc-review-portal/
■■■ README.md # Project overview & setup
■■■ FILE_EXPLANATIONS.txt # Detailed file documentation
■■■ package.json # Root workspace scripts
■
■■■ .env.example # Environment variable template
■
■■■ backend/ # Express.js API Server
■ ■■■ package.json # Backend dependencies
■ ■■■ tsconfig.json # TypeScript config
■ ■■■ prisma.config.ts # Prisma configuration
■ ■■■ prisma/
■ ■■■ schema.prisma # Database schema (781 lines)
■ ■■■ migrations/ # 16 migration folders
■ ■■■ src/
■ ■■■ server.ts # Express entry point
■ ■■■ app.ts # Express app factory
■ ■■■ config/
■ ■■■ branding.ts # Committee branding config
■ ■■■ prismaClient.ts # Prisma client singleton
■ ■■■ seed.ts # Database seeder (1,423 lines)
■ ■■■ seedReportsDemo.ts # Reports demo data seeder (469 lines)
■ ■■■ importCSV.ts # CSV import utilities
■ ■■■ routes/
■ ■■■ index.ts # Route barrel exports (9 route modules)
■ ■■■ healthRoutes.ts # Health check endpoints (33 lines)
■ ■■■ committeeRoutes.ts # Committee/panel endpoints (164 lines)
■ ■■■ dashboardRoutes.ts # Dashboard queues & filters (595 lines)
■ ■■■ projectRoutes.ts # Project CRUD & archives (840 lines)
■ ■■■ submissionRoutes.ts # Submissions/reviews (916 lines)
■ ■■■ mailMergeRoutes.ts # Letter generation (605 lines)
■ ■■■ importRoutes.ts # CSV import endpoints (435 lines)
■ ■■■ reportRoutes.ts # Academic year reports (266 lines)
■ ■■■ holidayRoutes.ts # Holiday CRUD (205 lines)
■ ■■■ services/
■ ■■■ letterGenerator.ts # DOCX letter builder (183 lines)
■ ■■■ imports/
■ ■■■ projectCsvImport.ts # CSV parsing & validation (580 lines)
■ ■■■ projects/
■ ■■■ createProjectWithInitialSubmission.ts # Project creation service (274 lines)
■ ■■■ reports/
■ ■■■ academicTerms.ts # Academic term helpers
■ ■■■ reportMetrics.ts # Report aggregation logic (406 lines)
■ ■■■ utils/
■ ■■■ index.ts
■ ■■■ csvUtils.ts # CSV escape utilities
■ ■■■ dashboardFilters.ts # Prisma where-clause builder (121 lines)
■ ■■■ holidayDate.ts # Holiday date helpers (37 lines)
■ ■■■ overdueClassifier.ts # Overdue severity classifier (140 lines)
■ ■■■ slaUtils.ts # Working days calculator
■ ■■■ workingDays.ts # Business day calculations (58 lines)
■ ■■■ middleware/
■ ■■■ auth.ts # Auth/session and role middleware
■ ■■■ generated/prisma/ # Generated Prisma client
■
■■■ frontend/ # React SPA
■ ■■■ package.json # Frontend dependencies
■ ■■■ index.html # Vite entry HTML (font preconnect)
■ ■■■ vite.config.ts # Vite bundler config
■ ■■■ tsconfig.json # TypeScript config
■ ■■■ src/
■ ■■■ App.tsx # React app root & routing
■ ■■■ main.tsx

# React entry point
■ ■■■ constants.ts # App-wide constants (SLA targets, thresholds, refresh intervals) (34 lines)
■ ■■■ components/ # Reusable UI components
■ ■■■ AttentionStrip.tsx
■ ■■■ Breadcrumbs.tsx # Navigation breadcrumbs

```

```

■ ■ ■■■ CommandBar.tsx # Top-bar search & actions
■ ■ ■■■ CsvDropzone.tsx # CSV file drag-and-drop
■ ■ ■■■ DashboardFilters.tsx # 5-filter bar (college, reviewType, etc.) (277 lines)
■ ■ ■■■ DashboardShell.tsx # Layout wrapper with sidebar (26 lines)
■ ■ ■■■ DashboardSidebar.tsx # Frosted-glass sidebar nav (127 lines)
■ ■ ■■■ EmptyState.tsx
■ ■ ■■■ ImportStepper.tsx # Multi-step import wizard
■ ■ ■■■ ImportSummary.tsx # Import results summary
■ ■ ■■■ LetterReadinessPanel.tsx
■ ■ ■■■ ProtocolProfileSection.tsx # Editable protocol profile (391 lines)
■ ■ ■■■ QueueTable.tsx # Queue data table (466 lines)
■ ■ ■■■ RowErrorsTable.tsx # Import error rows display
■ ■ ■■■ SLAStatusChip.tsx
■ ■ ■■■ SummaryCards.tsx
■ ■ ■■■ Timeline.tsx # Status history timeline
■ ■ ■■■ queue/ # Queue sub-components
■ ■ ■■■ QueueDataTable.tsx
■ ■ ■■■ QueueFilters.tsx
■ ■ ■■■ QueueKpiCards.tsx
■ ■ ■■■ hooks/ # Custom React hooks
■ ■ ■■■ useDashboardQueues.ts
■ ■ ■■■ useDashboardActivity.ts
■ ■ ■■■ useDashboardOverdue.ts
■ ■ ■■■ useProjectDetail.ts
■ ■ ■■■ useSubmissionDetail.ts
■ ■ ■■■ pages/
■ ■ ■■■ DashboardPageNew.tsx # Main dashboard (1,515 lines)
■ ■ ■■■ DashboardPage.tsx # Legacy dashboard (359 lines, unused)
■ ■ ■■■ QueuePage.tsx # Route-driven queue page (137 lines)
■ ■ ■■■ HolidaysPage.tsx # Calendar-based holiday mgmt (629 lines)
■ ■ ■■■ ReportsPage.tsx # Academic year reports (535 lines)
■ ■ ■■■ ImportProjectsPage.tsx # CSV import with preview (373 lines)
■ ■ ■■■ NewProtocolPage.tsx # Create protocol form, 24 fields (580 lines)
■ ■ ■■■ ArchivesPage.tsx # Archived projects with filters (321 lines)
■ ■ ■■■ ProjectDetailPage.tsx # Project detail & letters (528 lines)
■ ■ ■■■ SubmissionDetailPage.tsx # Submission detail & editing (1,042 lines)
■ ■ ■■■ LoginPage.tsx # Login page
■ ■ ■■■ SignupPage.tsx # Account request page
■ ■ ■■■ ChangePasswordPage.tsx # Password change page
■ ■ ■■■ services/
■ ■ ■■■ api.ts # API client (437 lines, 28+ functions)
■ ■ ■■■ styles/
■ ■ ■■■ globals.css # Design tokens & base styles (1,640 lines)
■ ■ ■■■ dashboard.css # Dashboard styles (3,487 lines)
■ ■ ■■■ imports.css # Import page styles (589 lines)
■ ■ ■■■ new-protocol.css # New protocol form (365 lines)
■ ■ ■■■ archives.css # Archives page + filters (383 lines)
■ ■ ■■■ protocol-profile.css # Protocol profile section (208 lines)
■ ■ ■■■ reports.css # Reports page (272 lines)
■ ■ ■■■ login.css # Login page styles (1,306 lines)
■ ■ ■■■ config/
■ ■ ■■■ branding.ts # Frontend branding config (30 lines)
■ ■ ■■■

■ types/
■ ■ ■■■ index.ts # TypeScript interfaces (616 lines)
■ ■ ■■■ utils/
■ ■ ■■■ dateUtils.ts # Working-day math & date formatting (54 lines)
■ ■ ■■■ overdueClassifier.ts # Frontend overdue logic (128 lines)
■ ■ ■■■ slaUtils.ts # Frontend SLA helpers (147 lines)
■ ■ ■■■

■■■ packages/shared/ # Shared constants, types & DTOs
■ ■ ■■■ package.json
■ ■ ■■■ src/
■ ■ ■■■ index.ts # Barrel export
■ ■ ■■■ constants.ts # Shared enums (roles, statuses, review types) (115 lines)
■ ■ ■■■ types.ts # Shared DTOs & interfaces (260 lines)
■ ■ ■■■

■■■ samples/ # Sample CSV exports
■ ■ ■■■ initial_ack.csv
■ ■ ■■■ initial_approval.csv
■ ■ ■■■

■■■ docs/
■■■ SECURITY.md # Security & RBAC documentation
■■■ user-manual.md # Operator handoff manual source
■■■ user-manual.pdf # Operator handoff manual PDF
■■■ neon-quota-runbook.md # Neon quota recovery runbook source
■■■ neon-quota-runbook.pdf # Neon quota recovery runbook PDF
■■■ fixes.md # Integration notes
■■■ reports-date-mapping.md # Reports date field mapping
■■■ reports-discovery-notes.md # Reports discovery notes
■■■ context-pack/ # This document
■■■ Project-Context-Pack.md # Editable project context pack
■■■ Project-Context-Pack.pdf # PDF project context pack

```

2. How to Run (Local Development)

Prerequisites

Requirement	Version	How to Check
Node.js	18+ (LTS)	<code>node -v</code>
npm	9+	<code>npm -v</code>
PostgreSQL	14+	<code>psql --version</code>

Environment Variables

Backend (backend/ .env):

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL connection string
PORT	No	Server port (default: 3000)
CORS_ORIGINS	No	Allowed origins (default: localhost:5173)
FRONTEND_URL	No	Frontend URL for CORS

Frontend (.env or Vite build):

Variable	Required	Description
VITE_API_URL	No	API base URL (default: http://localhost:3000)

Installation & Setup

```
# 1. Clone and enter repo
cd rerc-review-portal

# 2. Install all dependencies (root, backend, frontend)
npm install
npm run install:all

# 3. Create backend/.env with DATABASE_URL
echo 'DATABASE_URL=postgresql://USER@localhost:5432/rerc' > backend/.env

# 4. Ensure PostgreSQL is running and database exists
createdb rerc # or use pgAdmin/psql

# 5. Generate Prisma client and run migrations
npm run db:generate
npm run db:migrate

# 6. (Optional) Seed with sample data
npm run db:seed

# 7. Start development servers
npm run dev
```

Access Points

- **Backend API:** http://localhost:3000
- **Frontend UI:** http://localhost:5173 (default route redirects to /login)
- **Health check:** http://localhost:3000/health

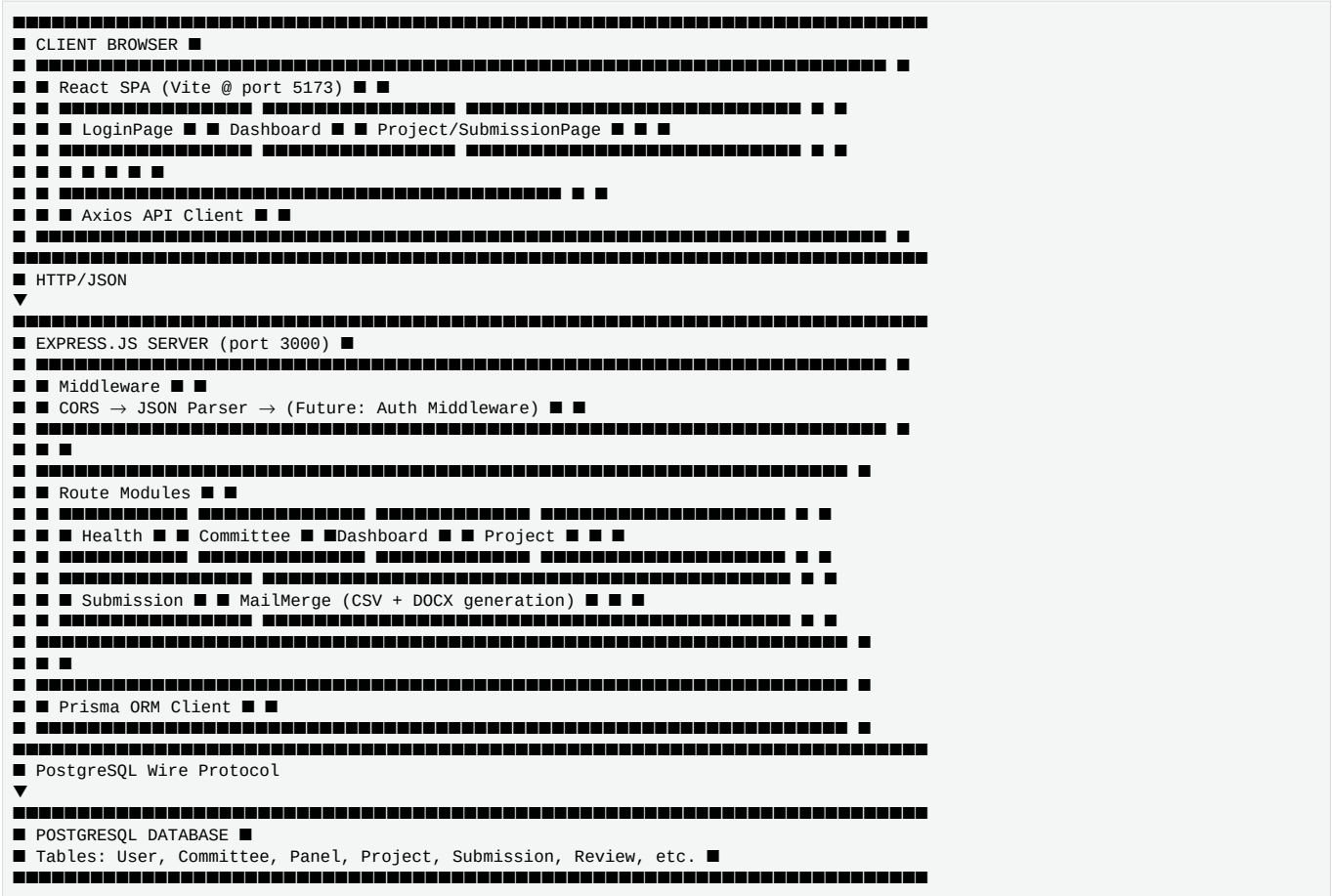
Common Failures & Fixes

Issue	Solution
P1001: Can't reach database	Check PostgreSQL is running, DATABASE_URL is valid
Cannot find module prisma	Run <code>npm run db:generate</code>
CORS errors in browser	Ensure CORS_ORIGINS includes frontend URL

Issue	Solution
Port 3000/5173 already in use	Kill existing process or change port in config
Missing CSV file during seed	Ensure [Intern Copy] RERC Protocol Database 2024 - 2024 Submission.csv exists at repo root

3. Architecture Overview

Component Diagram



Request Lifecycle (Client → API → DB)

1. User clicks "View Submission #5" in Dashboard
2. React Router renders SubmissionDetailPage
3. useSubmissionDetail hook fires
4. api.ts → GET /submissions/5
5. Express router matches submissionRoutes.ts
6. Handler calls prisma.submission.findUnique({ where: { id: 5 }, include: {...} })
7. Prisma generates SQL, executes against PostgreSQL
8. JSON response travels back through layers
9. React state updates, component re-renders

Auth Lifecycle

- Client Login/session cookie + CSRF-aware API
- calls through Axios

```
■  
▼  
  
■ authenticateUser middleware ■  
■ - Attaches req.user from authenticated session/token context ■  
■ - Supports explicit dev-header adapter only when enabled ■  
■ - Role guards and assignment guards authorize protected routes ■  
  
■  
▼  
  
■ csrfProtection + enforceForcedPasswordChange ■  
■ - Mutating requests require CSRF protection where applicable ■  
■ - Forced-password users can only use allowed auth/session paths ■
```

Production Recommendation: Review cookie settings, token/session expiry, CSRF behavior, and dev-header configuration before production rollout. Do not rely on frontend-only route guards for any sensitive permission.

4. Data Model (Database)

ERD Overview (Key Entities)

```

User ■■■■■■■■■■ Committee ■■■■■■■■■■ Panel ■■■■■■■■■■
■■■■■■■■■■ N:M via ■■■■■■■■■■ 1:N ■■■■■■■■■■
id ■■■■■■■■■■ Committee ■■■■■■■■■■ id ■■■■■■■■■■ id ■■■■■■■■■■
email ■■■■■■■■■■ Member ■■■■■■■■■■ code ■■■■■■■■■■ name ■■■■■■■■■■
fullName ■■■■■■■■■■ name ■■■■■■■■■■ committeeId ■■■■■■■■■■
roles[] ■■■■■■■■■■ isActive ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■
■■■■■■■■■■ 1:N ■■■■■■■■■■
■■■■■■■■■■ ▼ ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■ Project ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■ id ■■■■■■■■■■
■■■■■■■■■■ projectCode ■■■■■■■■■■
■■■■■■■■■■ title ■■■■■■■■■■
■■■■■■■■■■ piName ■■■■■■■■■■
■■■■■■■■■■ fundingType ■■■■■■■■■■
■■■■■■■■■■ overallStatus ■■■■■■■■■■
■■■■■■■■■■ committeeId ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■
■■■■■■■■■■ 1:N ■■■■■■■■■■
■■■■■■■■■■ ▼ ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■ Submission ■■■■■■■■■■ Classification ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ 1:1 ■■■■■■■■■■
■■■■■■■■■■ id ■■■■■■■■■■ reviewType ■■■■■■■■■■
■■■■■■■■■■ projectId ■■■■■■■■■■ panelId ■■■■■■■■■■
■■■■■■■■■■ submissionType ■■■■■■■■■■ classifiedById ■■■■■■■■■■
■■■■■■■■■■ receivedDate ■■■■■■■■■■
■■■■■■■■■■ status ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ finalDecision ■■■■■■■■■■
assignedTo ■■■■■■■■■■ createdById ■■■■■■■■■■
reviews ■■■■■■■■■■
■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■
■■■■■■■■■■ ▼ ▼ ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■ Review ■■■■■■■■■■ StatusHistory ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■■■■■■■■■■ reviewerId ■■■■■■■■■■ oldStatus ■■■■■■■■■■
■■■■■■■■■■ decision ■■■■■■■■■■ newStatus ■■■■■■■■■■
■■■■■■■■■■ remarks ■■■■■■■■■■ effectiveDate ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ changedById ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■

```

Key Enumerations

RoleType (User permissions):

- CHAIR — Committee chair, full committee access

- MEMBER — Committee/panel member, view-only for assigned
- RESEARCH_ASSOCIATE — RA/Secretariat, manages protocols
- RESEARCH_ASSISTANT — RA support, limited data entry
- REVIEWER — External reviewer, assigned submissions only
- ADMIN — System administrator, full access

SubmissionStatus (Workflow states):

1. RECEIVED — Initial receipt
2. UNDER_COMPLETENESS_CHECK — Documents being verified
3. AWAITING_CLASSIFICATION — Ready for review type decision
4. UNDER_CLASSIFICATION — Classification in progress
5. CLASSIFIED — Review type assigned
6. UNDER_REVIEW — Active reviewer assessment
7. AWAITING_REVISIONS — Revisions requested from PI
8. REVISION_SUBMITTED — PI resubmitted revisions
9. CLOSED — Final decision issued
10. WITHDRAWN — Applicant withdrew

ReviewType (Classification outcomes):

- EXEMPT — No further review needed
- EXPEDITED — Minimal-risk, expedited review
- FULL_BOARD — Full committee review required

ReviewDecision (Reviewer/final outcomes):

- APPROVED
- MINOR_REVISIONS
- MAJOR_REVISIONS
- DISAPPROVED
- INFO_ONLY

Newly added enum groups in latest schema migration:

- EndorsementStatus — Tracks endorsement workflow per review assignment
- ProjectMemberRole — Standardized roles for listed project members
- SubmissionDocumentType / SubmissionDocumentStatus — Required document tracking
- LetterDraftStatus — Draft lifecycle for generated letters
- ClassificationType / ReviewerRoundRole / DecisionStatus — Expanded governance and reviewer-round semantics

Critical Invariants (Code-Implied)

1. **Unique project codes:** `projectCode` has unique constraint, enforced by Prisma
2. **One classification per submission:** `submissionId` is unique on `Classification`
3. **Sequential submissions per project:** `@@unique([projectId, sequenceNumber])`
4. **Reviewer assignment uniqueness:** `@@unique([submissionId, reviewerId])` on `Review`

Migration History

Migration	Purpose
20251211100203_init_user	User model initialization
20251211103050_rerc_core	Core RERC models (Committee, Project)
20251211130711_add_classification	Classification model
20251211132529_add_reviews	Review and reviewer models
20251211134028_add_submission_status	Status history tracking
20251211135517_add_config_sla	SLA configuration tables
20251211142937_extend_sla_stages	Extended SLA stage enum
20251211150930_add_panel_member_and_deadlines	Panel membership model
20251211155533_add_submission_final_decision	Final decision fields
20251215021511_init	Re-initialization
20251215025323_init	Additional init
20260114130242_add_csv_fields	Fields for CSV import compatibility
20260210082226_update_schema	Major schema expansion (audit logs, review rounds, documents, decisions, status history, proponents)
20260211120000_reports_terms	AcademicTerm model for reports
20260212203000_allow_missing_protocol_fields	Make protocol fields nullable for partial data
20260212212000_protocol_profile_and_milestones	ProtocolProfile & ProtocolMilestone models

New Models (Since Feb 12)

ProtocolProfile — Denormalized protocol-level data (50+ fields) covering core info, reviewer assignments, honorarium status, progress/final/amendment/continuing report tracking. One-to-one with Project (projectId unique).

ProtocolMilestone — Ordered milestone entries for a project (label, days, dateOccurred, ownerRole, notes). Indexed by (projectId, orderIndex).

AcademicTerm — Academic year/term definitions (academicYear, term, startDate, endDate). Unique on (academicYear, term). Used by report endpoints for period-based aggregation.

Current RBAC and Assignment Data Paths

Feature	Storage Path	Who Can Assign/Manage	Resulting Access
Panel Management	Panel + PanelMember	Chair only	Chair can view/add/edit/delete panel members
Account approval	User.status, User.roles, approval metadata	Chair; Admin has limited account view/reset support	Approved users can sign in with assigned roles
Protocol assistant assignment	Submission.staffInChargeId	Chair, Research Associate	Assigned Research Assistant can see and operate that protocol
Reviewer assignment	Review.reviewerId	Chair, Research Associate	Assigned reviewer can access and submit review work

Important distinction:

- Submission.staffInChargeId is the **protocol assistant/operator** assignment.
- Review.reviewerId is the **review task** assignment.
- Research Assistants may see protocols through either path, but full protocol workflow controls are only granted when they are assigned through Submission.staffInChargeId.
- Backend enforcement lives primarily in backend/src/middleware/reviewerScope.ts.

Seed Script Notes

The seed script (backend/src/config/seed.ts, 1423 lines) reads CSV data from [Intern Copy] RERC Protocol Database 2024 - 2024 Submission.csv at the repo root and creates:

- Default committee (RERC-HUMAN)
- Users (RA, Chair, reviewers)
- Projects with submissions from CSV
- Classifications (107 records) with review type, panel, and reviewer assignments
- Populates piAffiliation and researchTypePHREB for all matched projects (132/133)

Known fix (Feb 15, 2026): Removed submissionType: "INITIAL" filter from findFirst query — existing submissions had null for that field, causing the upsert to silently skip all records.

5. API Surface

Route Inventory by Module

Health & Status

Method	Endpoint	Behavior	Auth
GET	/	Server status JSON	None
GET	/health	Database connection check	None

Committee Management

Method	Endpoint	Behavior	Auth
GET	/committees	List all committees with panels	None
GET	/committees/:code/panels	Get panels for a committee	None
GET	/panels/:id/members	Get panel members	None

Dashboard

Method	Endpoint	Behavior	Auth
GET	/dashboard/queues	Submission queues by status and role scope	Authenticated; Research Assistant scoped
GET	/dashboard/activity	Recent status change activity	Authenticated
GET	/dashboard/overdue	Overdue reviews and endorsements	Authenticated
GET	/dashboard/upcoming-due	Submissions with upcoming deadlines	Authenticated
GET	/dashboard/colleges	Distinct piAffiliation values for filter dropdown	Authenticated
GET	/ra/dashboard	Legacy/RA-specific dashboard data	Authenticated/scoped
GET	/ra/submissions/:id	Legacy/RA submission detail view	Authenticated/scoped

Admin, Account, and Panel Management

Method	Endpoint	Behavior	Auth Required
GET	/admin/users	List accounts for governance	CHAIR, ADMIN
GET	/admin/users/pending	List pending signup requests	CHAIR
POST	/admin/users/:id/approve	Approve signup and assign role	CHAIR
POST	/admin/users/:id/reject	Reject signup request	CHAIR
POST	/admin/users/:id/disable	Disable managed account	CHAIR
POST	/admin/users/:id/enable	Re-enable managed account	CHAIR

Method	Endpoint	Behavior	Auth Required
POST	/admin/users/:id/reset-password	Reset managed account password	CHAIR, ADMIN
GET	/admin/panel-management	SPA route for panel management	CHAIR
GET	/committees/:code/panels	Load committee panels	Authenticated
GET	/panels/:id/members	Load panel members	Authenticated

Project Management

Method	Endpoint	Behavior	Auth Required
POST	/projects	Create new project	CHAIR, RESEARCH_ASSOCIATE, ADMIN
GET	/projects	List all projects	Authenticated/scoped
GET	/projects/search	Search projects	Authenticated/scoped
GET	/projects/:id	Get project by ID	Authenticated/scoped
GET	/projects/:id/full	Get project with all relations	Authenticated/scoped
GET	/projects/:id/profile	Get protocol profile	Authenticated/scoped
PUT	/projects/:id/profile	Update protocol profile	Protocol operator access
POST	/projects/:id/profile/milestones	Create protocol milestone	Protocol operator access
PATCH	/projects/:id/profile/milestones/:mid	Update milestone	Protocol operator access
DELETE	/projects/:id/profile/milestones/:mid	Delete milestone	Protocol operator access
POST	/projects/:projectId/submissions	Create submission for project	CHAIR, RESEARCH_ASSOCIATE

Archives & Archived Projects

Method	Endpoint	Behavior	Auth Required
GET	/projects/archived	Fetch archived projects (CLOSED/WITHDRAWN). Filters: status, reviewType, college, search, page, pageSize	CHAIR, RESEARCH_ASSOCIATE, ADMIN
POST	/projects/with-submission	Create project with initial submission	CHAIR, RESEARCH_ASSOCIATE

Submission & Review

Method	Endpoint	Behavior	Auth Required
GET	/submissions/:id	Get submission with relations	Authenticated; assigned-only for Research Assistant/Reviewer
PATCH	/submissions/:id/status	Update submission status	Protocol operator access
PATCH	/submissions/:id/overview	Update submission overview	Protocol operator access
POST	/submissions/:submissionId/classifications	Add/update classification	CHAIR
GET	/submissions/reviewer-candidates	List reviewer candidates	CHAIR, RESEARCH_ASSOCIATE
GET	/submissions/assistant-candidates	List Research Assistant candidates	CHAIR, RESEARCH_ASSOCIATE
POST	/submissions/:submissionId/assistant-assignment	Assign one protocol assistant	CHAIR, RESEARCH_ASSOCIATE
POST	/submissions/bulk/assign-assistant	Assign one assistant to many protocols	CHAIR, RESEARCH_ASSOCIATE
POST	/submissions/:submissionId/reviews	Add reviewer assignment	CHAIR, RESEARCH_ASSOCIATE

Method	Endpoint	Behavior	Auth Required
POST	/submissions/bulk/assign-reviewer	Assign reviewer to many protocols	CHAIR, RESEARCH_ASSOCIATE
POST	/submissions/bulk/status-action	Bulk workflow action; classification actions Chair-only	CHAIR, RESEARCH_ASSOCIATE
GET	/submissions/:id/sla-summary	Get SLA deadline summary	Authenticated/scoped
POST	/reviews/:reviewId/decision	Submit review decision	Assigned reviewer / allowed operator
POST	/submissions/:id/final-decision	Record final committee decision	Protocol operator access

Mail Merge & Letter Generation

Method	Endpoint	Behavior	Auth
GET	/mail-merge/initial-ack.csv	Bulk acknowledgment CSV	None
GET	/mail-merge/initial-approval.csv	Bulk approval CSV	None
GET	/mail-merge/initial-ack/:submissionId	Single ack letter data	None
GET	/mail-merge/initial-ack/:submissionId/csv	Single ack CSV	None
GET	/mail-merge/initial-approval/:submissionId	Single approval letter data	None
GET	/mail-merge/initial-approval/:submissionId/csv	Single approval CSV	None
GET	/letters/initial-ack/:submissionId.docx	Generate DOCX ack letter	None
GET	/letters/initial-approval/:submissionId.docx	Generate DOCX approval letter	None

Project Import

Method	Endpoint	Behavior	Auth Required
POST	/api/imports/projects/preview	Preview CSV import mapping & validation	CHAIR, RESEARCH_ASSOCIATE
POST	/api/imports/projects	Import projects from CSV file	CHAIR, RESEARCH_ASSOCIATE
GET	/api/imports/projects/template	Download CSV template for import	CHAIR, RESEARCH_ASSOCIATE

Holiday Management

Method	Endpoint	Behavior	Auth Required
GET	/holidays	List holidays. Filters: year, from, to	ADMIN, CHAIR, RESEARCH_ASSOCIATE
POST	/holidays	Create holiday (409 if date exists)	ADMIN, CHAIR, RESEARCH_ASSOCIATE
PATCH	/holidays/:id	Update holiday (409 if duplicate date)	ADMIN, CHAIR, RESEARCH_ASSOCIATE
DELETE	/holidays/:id	Delete holiday	ADMIN, CHAIR, RESEARCH_ASSOCIATE

Reports

Method	Endpoint	Behavior	Auth Required
GET	/reports/academic-years	List available academic year/term options	None
GET	/reports/academic-year-summary	Aggregated metrics for selected term(s)	None

Background Jobs / Cron / Queues

None implemented. All operations are synchronous request-response. Potential future needs:

- SLA violation notification jobs
- Automated reminder emails
- Audit log archival

6. Frontend

Routing Structure

```
// frontend/src/App.tsx
<Routes>
  <Route path="/" element={<Navigate to="/login" />} />
  <Route path="/login" element={<LoginPage />} />
  <Route path="/signup" element={<SignupPage />} />
  <Route path="/change-password" element={<ProtectedRoute allowForcedPasswordChange><ChangePasswordPage /></ProtectedRoute>} />
  <Route path="/not-authorized" element={<NotAuthorizedPage />} />
  {/* DashboardShell provides sidebar + layout wrapper */}
  <Route element={<ProtectedRoute><DashboardShell /></ProtectedRoute>}>
    <Route path="/dashboard" element={<DashboardPageNew />} />
    <Route path="/admin/account-management" element={<ProtectedRoute allowedRoles={["CHAIR", "ADMIN"]} ><AdminAccountManagementPage /></ProtectedRoute>} />
    <Route path="/admin/panel-management" element={<ProtectedRoute allowedRoles={["CHAIR"]} ><PanelManagementPage /></ProtectedRoute>} />
    <Route path="/queues/:queueKey" element={<QueuePage />} />
    <Route path="/queues/exempted" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE"]} ><ExemptedPage /></ProtectedRoute>} />
    <Route path="/calendar" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE", "ADMIN"]} ><CalendarPage /></ProtectedRoute>} />
    <Route path="/account/profile" element={<MyProfilePage />} />
  </Route>
  <Route path="/projects/new" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE"]} ><NewProtocolPage /></ProtectedRoute>} />
  <Route path="/projects/new-classic" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE"]} ><NewProtocolClassicPage /></ProtectedRoute>} />
  <Route path="/imports/projects" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE"]} ><ImportProjectsPage /></ProtectedRoute>} />
  <Route path="/reports" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE"]} ><ReportsPage /></ProtectedRoute>} />
  <Route path="/archives" element={<ProtectedRoute allowedRoles={["CHAIR", "RESEARCH_ASSOCIATE", "ADMIN"]} ><ArchivesPage /></ProtectedRoute>} />
  <Route path="/recently-deleted" element={<ProtectedRoute allowedRoles={["CHAIR", "ADMIN"]} ><RecentlyDeletedPage /></ProtectedRoute>} />
  <Route path="/projects/:projectId" element={<ProtectedRoute><ProjectDetailPage /></ProtectedRoute>} />
  <Route path="/submissions/:submissionId" element={<ProtectedRoute><SubmissionDetailPage /></ProtectedRoute>} />
</Routes>
```

Key Pages & Their API Calls

Page	Purpose	API Endpoints Used
LoginPage	User authentication	Auth/session endpoints
SignupPage	Account request creation	Auth/signup endpoint
ChangePasswordPage	Forced or user-initiated password change	Auth/password endpoint
AdminAccountManagementPage	Chair/Admin account governance	/admin/users, approval/reject/disable/enable/reset endpoints
PanelManagementPage	Chair-only panel member management	/committees/:code/panels, /panels/:id/members, panel member mutation endpoints
NewProtocolPage	Create new project with submission (24 fields, 2-step flow)	POST /projects/with-submission, GET /committees
ImportProjectsPage	Bulk CSV project/submission import	POST /api/imports/projects/preview, POST /api/imports/projects, GET /api/imports/projects/template
ArchivesPage	View archived/completed projects (3 filters: Status, Review Type, College)	GET /projects/archived, GET /dashboard/colleges
RecentlyDeletedPage	Soft-delete recovery and permanent deletion area	Project soft-delete endpoints

Page	Purpose	API Endpoints Used
DashboardPageNew	Main dashboard with route-driven queues, bulk actions, quick view, assistant/reviewer assignment	/dashboard/queues, /dashboard/activity, /dashboard/overdue, /dashboard/colleges, /projects/search, assignment endpoints
QueuePage	Individual queue view; Research Assistant sees assigned records only	/dashboard/queues (filtered by queue key and role scope)
ExemptedPage	Exempted protocol queue	Exempted queue endpoints
CalendarPage	Calendar-based holiday/deadline management	GET/POST/PATCH/DELETE /holidays, deadline data
ReportsPage	Academic year summary reports	/reports/academic-years, /reports/academic-year-summary
ProjectDetailPage	Project details, protocol profile & letter export	/projects/:id/full, /projects/:id/profile, /mail-merge/*, /letters/*
SubmissionDetailPage	Submission details, editing, timeline, assistant assignment, reviewer assignment, review decision area	/submissions/:id, /submissions/:id/sla-summary, /committees, PATCH /submissions/:id/overview, assignment/review endpoints

State Management Approach

- **No global state library** (no Redux, Zustand, etc.)
- **React hooks pattern:** Custom hooks (useDashboardQueues, useSubmissionDetail, etc.) encapsulate data fetching with useState/useEffect
- **URL-based state:** Filter selections stored in URL query params
- **Local storage:** Dashboard preferences (collapsed panels, density)

Forms & Validation Strategy

- **Controlled inputs:** Form state managed via useState
- **Client-side validation:** Basic required field checks before submission
- **Server-side validation:** Express handlers validate enum values, required fields
- **No form library:** No Formik, React Hook Form, or Zod integration

7. Security & Privacy Checklist

Authentication Method

Aspect	Current State	Risk Level
Auth mechanism	Auth routes with authenticated request middleware; dev-header adapter only when enabled	MEDIUM
Session management	Session/cookie middleware and forced-password-change gate exist	MEDIUM
Password storage	passwordHash field and managed password reset flow exist	MEDIUM
CSRF	csrfProtection middleware is mounted after auth	MEDIUM
Rate limiting	globalLimiter is mounted	MEDIUM

RBAC/Authorization Approach

- **Authentication middleware:** authenticateUser is mounted globally before protected route modules.
- **Role definitions:** CHAIR, MEMBER, RESEARCH_ASSOCIATE, RESEARCH_ASSISTANT, REVIEWER, ADMIN.
- **Frontend route guards:** ProtectedRoute controls visible page access and redirects unauthorized users to /not-authorized.

- **Sidebar guards:** DashboardSidebar hides role-inappropriate navigation such as Chair-only Panel Management.
- **Backend role guards:** requireRole and requireAnyRole protect role-specific mutations.
- **Backend assignment guards:** requireSubmissionAccess, requireProjectAccess, requireSubmissionOperatorAccess, and requireProjectOperatorAccess enforce assigned-only Research Assistant behavior.
- **Panel Management:** Chair-only page and navigation. Panel member mutations must remain Chair-only.
- **Protocol assistant assignment:** Chair/Research Associate only. Stored as Submission.staffInChargeId.
- **Reviewer assignment:** Chair/Research Associate only. Stored as Review.reviewerId.
- **Research Assistant access:** Can view assigned protocols; protocol workflow operation requires staff-in-charge assignment.

Input Validation & Sanitization

Location	Validation Type
Route handlers	Basic required field checks, enum validation
CSV import (seed.ts)	safeTrim, date parsing, enum mapping
CSV export (mailMergeRoutes)	csvEscape for special characters
Frontend forms	Controlled inputs, basic validation

Gap: No comprehensive XSS/injection sanitization library (e.g., DOMPurify, validator.js)

File Upload / CSV Import Threat Notes

- **CSV seed parsing:** Reads external file directly, minimal sanitization
- **Document links:** Stored as URLs (e.g., Google Drive), not file uploads
- **DOCX generation:** Built server-side with docx library, controlled content
- **No file upload endpoint exists** (future risk if added)

Logging/Auditing Approach

- **AuditLog model exists** in Prisma schema.
- **Structured request logging:** pino-http is mounted with request IDs.
- **Request IDs:** requestId middleware attaches IDs for traceability.
- **Error handling:** centralized errorHandler is mounted last.
- **Remaining gap:** operational audit events should be reviewed for complete coverage of sensitive account, assignment, and protocol workflow changes.

Obvious Secret Exposure Risks

- No hardcoded secrets found in source code
- DATABASE_URL must be in .env (not checked into repo)
- .env.example exists at repo root with placeholder values for DATABASE_URL, PORT, CORS_ORIGINS, FRONTEND_URL

8. Testing & Quality

Existing Tests

Path	Type	Description
backend/tests/unit/projectCsvImport.test.ts	Unit	CSV parsing and validation tests

Path	Type	Description
backend/tests/unit/overdueClassifier.test.ts	Unit	Overdue severity classification tests
backend/tests/unit/dashboardFilters.test.ts	Unit	Prisma where-clause builder tests
backend/tests/unit/workingDays.test.ts	Unit	Business day calculation tests
backend/tests/unit/academicTerms.test.ts	Unit	Academic term helper tests
backend/tests/unit/reportMetrics.test.ts	Unit	Report aggregation logic tests
backend/tests/unit/holidayDate.test.ts	Unit	Holiday date helper tests
backend/tests/integration/importProjects.test.ts	Integration	Project CSV import flow tests
backend/tests/integration/projectCreate.test.ts	Integration	Project creation service tests
backend/tests/api/holidayRoutes.test.ts	API	Holiday CRUD endpoint tests
backend/tests/api/reportRoutes.test.ts	API	Report endpoint tests

How to Run Tests

```
cd backend
npm test # Run all tests
npm run test:unit # Unit tests only
npm run test:integration # Integration tests
npm run test:api # API tests
```

Lint/Format Hooks

- **ESLint:** Configured for frontend (frontend/package.json has lint script)
- **Prettier:** Not found in config
- **Husky/lint-staged:** Not configured (no pre-commit hooks)

Coverage Gaps (Inferred)

1. **No frontend tests:** No test files in frontend/src
2. **No E2E tests:** No Playwright, Cypress, or similar
3. **Limited API test coverage:** Only security tests found
4. **No CI pipeline:** Tests not automated in GitHub Actions/similar

9. Current Known Issues / TODO Extraction

TODO/FIXME Comments

File	Line	Comment
projectRoutes.ts	59	TODO: replace with real logged-in user later
submissionRoutes.ts	201	TODO: replace with authenticated user later
submissionRoutes.ts	511	TODO: replace with authenticated user later
LoginPage.tsx	121	TODO: Replace with actual API call once backend auth is implemented
ForgotPasswordPage.tsx	20	TODO: Replace with actual API call

Likely Fragile Modules

Module	Reason	File Reference
DashboardPageNew.tsx	1,515 lines, complex state, should be split	frontend/src/pages/DashboardPageNew.tsx
SubmissionDetailPage.tsx	1,042 lines, heavy editing logic	frontend/src/pages/SubmissionDetailPage.tsx
submissionRoutes.ts	916 lines, multiple responsibilities	backend/src/routes/submissionRoutes.ts
projectRoutes.ts	840 lines, project + archive + profile routes	backend/src/routes/projectRoutes.ts
seed.ts	1,423 lines, complex CSV parsing	backend/src/config/seed.ts
Auth middleware	Exists but not consistently wired	docs/SECURITY.md describes design
SLA calculations	Simple working-days only, holiday table exists but unused in SLA	backend/src/utis/slaUtils.ts

Technical Debt Observations

1. **Large files:** Multiple files exceed 500 lines, violating single-responsibility
2. **Auth incomplete:** Header-based auth is insecure, needs production implementation
3. **Hardcoded IDs:** createdById: 1 used throughout
4. **No API versioning:** Routes at root, no /api/v1 prefix
5. **Mixed concerns:** Route files contain business logic, should extract to services
6. **No error handling middleware:** Individual try/catch in each handler

10. Consultant-Ready "Next Questions"

High Priority (Security/Architecture)

1. **What authentication provider will production use?** (LDAP, OAuth, custom JWT?)
2. **What is the user provisioning process?** (Manual creation, SSO sync?)
3. **Are there compliance requirements?** (Data retention, audit trail mandates?)
4. **What is the expected concurrent user load?** (Sizing for database/server)
5. **Is there an existing deployment infrastructure?** (On-prem, cloud, containerized?)

Medium Priority (Functionality)

6. **How should SLA deadlines handle holidays?** (The Holiday table exists but isn't used)
7. **What email service will send notifications?** (SMTP, SendGrid, etc.)
8. **Are there other committees beyond RERC-HUMAN?** (Multi-committee support is modeled)
9. **What reports do chairs/admins need beyond the dashboard?**
10. **How should the letter templates be managed?** (Hardcoded vs. admin-configurable)

Data & Integration

11. **Is there existing data to migrate beyond the sample CSV?**
12. **Does this system need to integrate with other university systems?** (HRMS, LMS, etc.)
13. **What is the document storage strategy?** (Currently just URLs, need upload?)
14. **How should continuing reviews be scheduled and reminded?**
15. **What happens when a project approval period expires?**

Process Clarification

16. **What is the exact workflow for full-board reviews?** (Meeting scheduling, voting)
17. **How are reviewer honoraria processed?** (Model exists, process unclear)
18. **What defines "completeness" for a submission?** (Checklist, documents?)
19. **Who can reassign submissions to different staff?**
20. **What audit records must be retained and for how long?**

Appendix A: Quick Reference Commands

```
# Start development (both servers)
npm run dev

# Backend only
npm run dev:backend

# Frontend only
npm run dev:frontend

# Database commands
npm run db:generate # Generate Prisma client
npm run db:migrate # Run migrations
npm run db:seed # Seed database

# Open Prisma Studio (DB GUI)
cd backend && npx prisma studio

# Run tests
cd backend && npm test
```

Appendix C: Recent Changes

February 15, 2026

Premium UI Overhaul

- **Design Token System** — 4-level shadow elevation (--shadow-xs/sm/md/lg), semantic badge tokens (20+ hardcoded colors extracted to --badge-* -bg/fg/border and --role-* -bg/fg variables), --spacing-* scale
- **Font Preconnect** — Added Google Fonts preconnect links in index.html; removed font-family override from dashboard CSS
- **Frosted-Glass Sidebar** — backdrop-filter: blur() on DashboardSidebar
- **Hover Elevation** — Cards gain shadow depth on hover
- **Skeleton Shimmer** — Re-enabled loading shimmer animations
- **Staggered Entrance Animations** — Dashboard cards fade in with cascading delays
- **3-Tier Click Feedback** — :active press states on 30+ interactive elements (buttons, cards, pagination, etc.) with scale values: small 0.90–0.96, medium 0.97, large 0.985, full-width: background darken
- **Button Normalize** — :active scale + consistent transition tokens across all button variants

Dashboard Filters Fix

- **Root Cause:** Database had NULL values for piAffiliation, researchTypePHREB, and zero Classification records. Seed script's findFirst searched for submissionType: "INITIAL" but existing submissions had null submissionType.
- **Fix:** Removed submissionType condition from seed's findFirst query. Re-seeded: 132/133 projects populated, 107 classifications created.
- **College Filter Upgrade:** Converted college text input to <select> dropdown populated by new GET /dashboard/colleges endpoint. Added case-insensitive matching in dashboardFilters.ts.

Archive Page Filters

- Added 3 server-side filter dropdowns to Archives page: **Status** (Closed/Withdrawn), **Review Type** (Exempt/Expedited/Full Board), **College** (dynamic from DB)
- "Clear filters" button appears when any filter is active
- Backend GET /projects/archived now accepts status, reviewType, college query params
- College uses case-insensitive match; ReviewType filters through classification relation

Files Changed (14 files, 469 insertions, 120 deletions)

File	Change
frontend/index.html	Added font preconnect links
frontend/src/App.tsx	Copyright year 2025→2026
frontend/src/styles/globals.css	Shadow system, badge tokens, spacing scale, :active states
frontend/src/styles/dashboard.css	Frosted sidebar, shimmer, staggered animations, hover elevation, press states
frontend/src/styles/archives.css	Filter row, dropdown, clear button, responsive styles
frontend/src/styles/protocol-profile.css	Transition normalization
frontend/src/styles/new-protocol.css	Transition normalization
frontend/src/components/DashboardFilters.tsx	College text→dropdown, fetchColleges() import
frontend/src/pages/ArchivesPage.tsx	3 filter dropdowns, clear button
frontend/src/services/api.ts	fetchColleges(), updated fetchArchivedProjects params
backend/src/config/seed.ts	Removed submissionType: INITIAL filter
backend/src/routes/dashboardRoutes.ts	Added GET /dashboard/colleges endpoint
backend/src/routes/projectRoutes.ts	Archive filters: status, reviewType, college
backend/src/utils/dashboardFilters.ts	Case-insensitive college filter

February 13, 2026

Dashboard Queue Redesign

- **Route-driven queues** — Dashboard refactored from single mega-page to DashboardShell layout with DashboardSidebar + routed sub-pages (/queues/:queueKey)
- **DashboardShell.tsx** — New layout wrapper component (26 lines) using <Outlet>
- **DashboardSidebar.tsx** — Frosted-glass sidebar with queue navigation links (127 lines)
- **QueuePage.tsx** — Individual queue view page (137 lines)
- **DashboardPageNew.tsx** reduced from 2335→1515 lines through component extraction
- **Pagination** — PAGE_SIZE=15 constant, stable table height with min-height: 780px

Protocol Profile Shared Component

- **ProtocolProfileSection.tsx** — New shared component (375 lines) for in-place editing of protocol profile fields
- Groups fields into "Core Information" and "Panel & Reviewers" with collapsible sections
- Supports inline editing with save/cancel, milestone CRUD
- Used in both ProjectDetailPage and SubmissionDetailPage

Create Protocol Form Expansion

- **NewProtocolPage.tsx** expanded from 331→580 lines
- Now covers 24 fields in a 2-step flow (basic info → detailed fields)
- Added fields: department, piAffiliation, researchTypePHREB, funding type, and more

Holidays Page

- **HolidaysPage.tsx** — New calendar-based holiday management page (629 lines)
- Full calendar grid with month navigation
- Inline CRUD: create, edit, delete holidays directly on the calendar
- Route: /holidays (inside DashboardShell)

Reports Page

- **ReportsPage.tsx** — Academic year summary reports page (535 lines)
- Backend: reportRoutes.ts (266 lines), academicTerms.ts, reportMetrics.ts
- New data model: AcademicTerm (migration 20260211120000_reports_terms)

Protocol Profile & Milestones Data Model

- **ProtocolProfile** model — 50+ fields for denormalized protocol data (migration 20260212212000)
- **ProtocolMilestone** model — Ordered milestone tracking per project
- GET /projects/:id/profile, PUT /projects/:id/profile
- POST/PATCH/DELETE /projects/:id/profile/milestones/:mid

New API Functions

- fetchProtocolProfile(), updateProtocolProfile()
- createProtocolMilestone(), updateProtocolMilestone(), deleteProtocolMilestone()
- fetchHolidays(), createHoliday(), updateHoliday(), deleteHoliday()
- fetchReportAcademicYears(), fetchAcademicYearSummary()

New TypeScript Types

- ProtocolProfile, UpdateProtocolProfilePayload
- ProtocolMilestone, CreateProtocolMilestonePayload, UpdateProtocolMilestonePayload
- HolidayItem, CreateHolidayPayload, UpdateHolidayPayload
- ReportsAcademicYearOption, ReportsSummaryResponse

New Tests

- backend/tests/api/holidayRoutes.test.ts — Holiday CRUD endpoint tests
- backend/tests/api/reportRoutes.test.ts — Report endpoint tests
- backend/tests/unit/holidayDate.test.ts — Holiday date helper tests
- backend/tests/unit/academicTerms.test.ts — Academic term tests
- backend/tests/unit/reportMetrics.test.ts — Report metrics tests

February 10, 2026

New Features

- **New Protocol Page** (/projects/new) — Form to create new projects with initial submission
- **Archives Page** (/archives) — View archived/completed projects (CLOSED/WITHDRAWN status)
- **Project CSV Import Preview** — Two-step import flow with preview and column mapping validation
- **Archive API Endpoint** — GET /projects/archived for fetching completed protocols

New Components

- `RowErrorsTable.tsx` — Display import validation errors by row/field

New Backend Services

- `backend/src/services/imports/projectCsvImport.ts` — CSV parsing, validation, and column mapping (534 lines)
- `backend/src/services/projects/createProjectWithInitialSubmission.ts` — Service for creating projects with initial submission (180 lines)

New API Functions (Frontend)

- `createProjectWithInitialSubmission()` — Create project with first submission
- `fetchArchivedProjects()` — Fetch archived/completed projects with pagination and search

New TypeScript Types

- `ImportRowError`, `ImportResult`, `ProjectImportPreview` — Import-related types
- `CreateProjectPayload`, `CreateProjectResponse` — Project creation types
- `ArchivedProject`, `ArchivedProjectsResponse` — Archive page types

New Tests

- `backend/tests/unit/projectCsvImport.test.ts` — CSV parsing unit tests
- `backend/tests/integration/importProjects.test.ts` — Import flow integration tests
- `backend/tests/integration/projectCreate.test.ts` — Project creation integration tests

Files Changed/Added

File	Change
<code>frontend/src/pages/NewProtocolPage.tsx</code>	New page for creating protocols (331 lines)
<code>frontend/src/pages/ArchivesPage.tsx</code>	New page for viewing archives (261 lines)
<code>frontend/src/components/RowErrorsTable.tsx</code>	New component for import errors
<code>frontend/src/styles/new-protocol.css</code>	Styles for new protocol form
<code>frontend/src/styles/archives.css</code>	Styles for archives page
<code>frontend/src/services/api.ts</code>	Added <code>createProjectWithInitialSubmission</code> , <code>fetchArchivedProjects</code>
<code>frontend/src/types/index.ts</code>	Added archive and import types
<code>frontend/src/App.tsx</code>	Added routes for <code>/projects/new</code> and <code>/archives</code>
<code>backend/src/routes/importRoutes.ts</code>	Added <code>/preview</code> endpoint for CSV import
<code>backend/src/routes/projectRoutes.ts</code>	Added <code>/projects/archived</code> and <code>/projects/with-submission</code>
<code>backend/src/services/imports/projectCsvImport.ts</code>	New CSV import service
<code>backend/src/services/projects/createProjectWithInitialSubmission.ts</code>	New project creation service

February 11, 2026

Data Model Expansion (Documented Late)

- Added migration: `backend/prisma/migrations/20260210082226_update_schema`
- Expanded workflow support with new models for:
 - `ClassificationVote`
 - `ProjectStatusHistory`
 - `ReviewAssignment`

- SubmissionDecision
- SubmissionDocument
- Added project governance/history models:
- ProjectMember
- ProjectChangeLog
- ProjectSnapshot
- SubmissionChangeLog
- Added proponent linkage models:
- Proponent
- ProjectProponent

Prisma Client Regeneration

- Generated Prisma client files under backend/src/generated/prisma/ were updated to reflect the new schema.
- New generated model files include:
- backend/src/generated/prisma/models/ClassificationVote.ts
- backend/src/generated/prisma/models/ProjectStatusHistory.ts

Dependency/Lockfile Updates

- backend/package-lock.json updated with backend schema/client-related dependency state.
- frontend/package-lock.json updated as part of local dependency synchronization.

February 12, 2026

Legacy CSV Compliance Direction (Approved; Implementation In Progress)

- **Compliance target selected:** normalized schema + computed/derived metrics (not literal 1:1 denormalized storage).
- **Legacy ambiguity policy:** preserve raw legacy values when mapping is uncertain.
- **Unknown status handling:** Unknown maps to safe normalized status and is flagged so it does not contaminate operational dashboards by default.
- **Deterministic clearance anchor:** clearance/issuance date is persisted as a workflow event (CLEARANCE_ISSUED).

Planned Schema Deltas (PR Spec Locked)

- Add Submission.isLegacyAmbiguous (Boolean, default false).
- Add Submission.legacyStatusRaw and Submission.legacyReviewTypeRaw (nullable strings).
- Add FollowUpRequest entity for post-approval tracking (Progress/Final/Amendment/Continuing) with cycle support.
- Add/extend enums for legacy mapping compatibility:
- FundingType.OTHER
- ProponentCategory.STAFF
- WorkflowEventType.CLEARANCE_ISSUED
- ReviewerRoundRole.INDEPENDENT_CONSULTANT
- Make Holiday.date date-only storage (@db.Date) and keep unique by date.

Idempotent Import Keys (Must-Have)

- WorkflowEvent: unique (submissionId, eventType, cycleNumber).

- **FollowUpRequest:** unique (projectId, type, cycleNumber).
- **ReviewAssignment:** unique (submissionId, roundSequence, reviewerRole) used for deterministic role-slot upserts.

Importer Behavior (Planned)

- Canonical header normalization for messy CSV headers (newline cleanup, dedupe, context-aware # days disambiguation).
- Table-driven value mapping for Status, Funding, ProponentCategory, and ResearchTypePHREB.
- Per-row transactional upsert with continue-on-error batch processing.
- Import report emits inserted/updated/failed counts and row-level reasons.

Reporting/Dashboard Guardrail (Planned)

- Dashboard and report queries will exclude isLegacyAmbiguous=true by default.
- Audit flows can opt in to include ambiguous legacy records explicitly.

February 9, 2026

New Features

- **CSV Import Page** (/imports/projects) — Bulk import projects and submissions from CSV
- **Breadcrumbs Component** — Added navigation breadcrumbs for better wayfinding
- **Back Button Navigation** — Import page now has "Back to Dashboard" link

Bug Fixes

- **FormData Content-Type Fix** — Fixed CSV file upload by explicitly setting multipart/form-data header in axios request (was overriding with application/json)

Files Changed

File	Change
frontend/src/components/Breadcrumbs.tsx	New component for navigation breadcrumbs
frontend/src/components/index.ts	Export Breadcrumbs component
frontend/src/pages/ImportProjectsPage.tsx	Added breadcrumbs and back link
frontend/src/services/api.ts	Fixed importProjectsCsv Content-Type header
frontend/src/styles/imports.css	Added breadcrumb and back-link styles

May 28, 2026

RBAC and Handoff Documentation Update

- **User manual added:** docs/user-manual.md and docs/user-manual.pdf now document the real operator workflow for Chairs, Research Associates, Research Assistants, reviewers, and admins.
- **Neon quota runbook added:** docs/neon-quota-runbook.md and docs/neon-quota-runbook.pdf explain what to do if compute, storage, or network transfer usage approaches or exceeds the Neon Free plan allowance.
- **Project context pack refreshed:** this document and Project-Context-Pack.pdf were updated for the current RBAC and assignment behavior.
- **Panel Management documented:** Chair-only sidebar item and route at /admin/panel-management.
- **Account Management documented:** account approval, role assignment, rejection, disable/enable, and reset-password workflows.
- **Assistant assignment documented:** Chair/Research Associate can assign a Research Assistant as protocol assistant. This stores Submission.staffInChargeId and grants protocol-operator access for that assigned protocol.
- **Reviewer assignment separation documented:** Assign reviewer continues to create review work through Review.reviewerId and does not grant protocol-operator access.

- **Research Assistant assigned-only rule documented:** Research Assistants only see assigned protocols and cannot directly access unassigned protocols through URL/API access.

Files Changed/Added

File	Change
docs/user-manual.md	Full editable user manual source
docs/user-manual.pdf	PDF handoff manual
docs/neon-quota-runbook.md	Neon database quota operations runbook
docs/neon-quota-runbook.pdf	PDF copy of Neon quota operations runbook
docs/context-pack/Project-Context-Pack.md	Updated source project context pack
docs/context-pack/Project-Context-Pack.pdf	Regenerated PDF context pack
FILE_EXPLANATIONS.txt	Updated RBAC handoff and file map
README.md	Added user manual links

Appendix D: Security Checklist Confirmation

- [x] **No real secrets/tokens in this document**
- [x] **DATABASE_URL shown as placeholder only**
- [x] **No real user emails or passwords exposed**
- [x] **Noted security gaps are design concerns, not exploits**

Document updated May 28, 2026 by analyzing repository structure, source files, RBAC implementation, user manual handoff materials, and documentation. Some details are inferred where explicit documentation was incomplete.